AgroPlus

Documentação técnica: principais funções e banco de dados

React Native 0.86 · Expo SDK 57 · TypeScript · Firebase Auth · Cloud Firestore

Davi Ugioni Marcolino — Projeto Integrador, Curso de Informática
Novembro de 2026

Sumário

1. Visão geral da arquitetura
2. O banco de dados (Cloud Firestore)
3. Regras de segurança
4. Índices
5. Autenticação e conta
6. Serviços: Propriedades e Talhões
7. Serviços: Despesas
8. Serviços: Plantios e estimativa de colheita
9. Resumo e cache
10. Arquivos locais (fotos e notas fiscais) e preferências
11. Exportação para planilha Excel
12. Tratamento de erros
13. Fluxos completos (passo a passo)

1. Visão geral da arquitetura

O AgroPlus é um app mobile para o produtor rural registrar propriedades, talhões, despesas e plantios. Ele não tem um servidor próprio: o app conversa direto com o **Firebase**, que oferece a autenticação (**Firebase Auth**) e o banco de dados na nuvem (**Cloud Firestore**). A segurança, portanto, fica nas **regras do Firestore**, que são avaliadas no servidor do Google a cada leitura ou escrita.

O código é organizado em camadas:

Pasta / arquivo	Responsabilidade
`Screens/`	Telas (Login, Home, Propriedades, Despesas, Plantios, Resumo, Configurações...). Cuidam da interface e validação dos formulários.
`Services/`	Toda a lógica de acesso a dados: funções que leem e gravam no Firestore, cálculos de totais, arquivos locais e exportação.
`Components/`	Componentes visuais reutilizáveis (botões, campos, cartões, gráficos, toasts, confirmações).
`constants/`	Dados fixos: catálogo de culturas e seus ciclos, estilo do mapa.
`FirebaseConfig.ts`	Inicializa o Firebase e exporta `db` (Firestore) e `auth` (Auth com sessão persistente via AsyncStorage).
`firestore.rules`	Regras de segurança do banco.
`firestore.indexes.json`	Índices compostos do banco.
`App.tsx`	Navegação: pilha de telas + barra inferior com 5 abas (Home, Propriedades, Plantios, Despesas, Resumo). Observa o login com `onAuthStateChanged`.

Regra de ouro do projeto: **as telas nunca chamam o Firestore diretamente**; elas chamam funções de `Services/`. Assim, a lógica de dados fica num único lugar e pode ser reaproveitada (por exemplo, a Home, o Resumo e a exportação usam as mesmas funções de listagem).

Conexão com o Firebase

```
const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
// Auth com persistência: o usuário continua logado ao fechar o app
export const auth = initializeAuth(app, {
  persistence: getReactNativePersistence(AsyncStorage),
});
```

O arquivo real `FirebaseConfig.ts` fica fora do Git (.gitignore); o repositório traz só o modelo `FirebaseConfig.example.ts`.

2. O banco de dados (Cloud Firestore)

O Firestore é um banco **NoSQL orientado a documentos**. Em vez de tabelas e linhas, há **coleções** com **documentos** (objetos JSON com um ID). Não existem JOINs nem chaves estrangeiras automáticas: os relacionamentos são feitos guardando o ID do documento "pai" num campo do "filho".

O AgroPlus usa **4 coleções na raiz** do banco. Todas são "planas" (não há subcoleções); a ligação entre elas é feita pelos campos `userId`, `propriedadeId` e `talhaoId`.

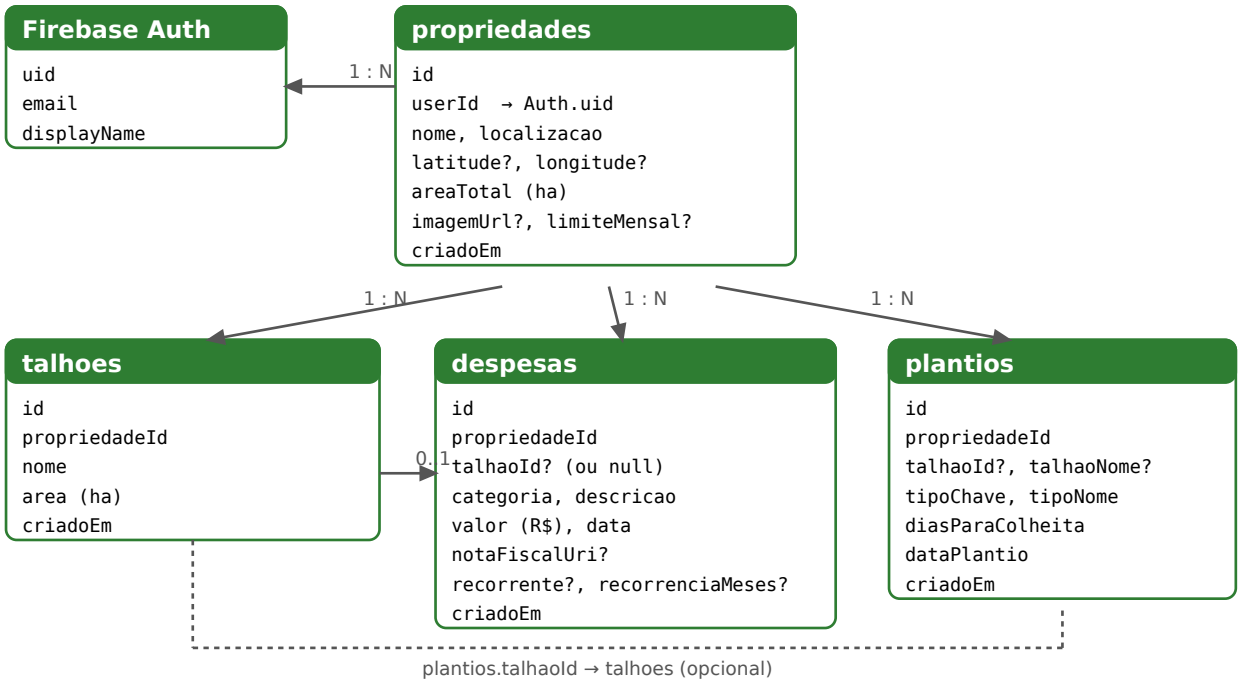


Figura 1 — Coleções e relacionamentos. A seta aponta para o documento referenciado.

Hierarquia lógica: **Usuário → Propriedade → (Talhões, Despesas, Plantios)**. Despesas e plantios pertencem sempre a uma propriedade e, opcionalmente, a um talhão dela.

Coleção propriedades

Campo	Tipo	Descrição
userId	string	UID do dono (Firebase Auth). Base de toda a segurança.
nome	string	Nome da fazenda/sítio.
localizacao	string	Endereço em texto (digitado ou obtido do GPS/mapa).
latitude, longitude	number?	Coordenadas marcadas no mapa.
areaTotal	number	Área total em hectares; limita a soma dos talhões.
imagemUrl	string?	URI da foto **local** no aparelho (não vai para a nuvem).
limiteMensal	number?	Campo previsto no modelo para limite de gasto mensal.
exemplo	boolean?	Marca propriedades criadas pelo gerador de dados de exemplo.
criadoEm	timestamp	Data de criação gerada no servidor (serverTimestamp).

Coleção talhoes

Campo	Tipo	Descrição
propriedadeId	string	ID da propriedade à qual o talhão pertence.
nome	string	Ex.: "Área Norte", "Talhão 3".
area	number	Área em hectares.
criadoEm	timestamp	Data de criação (serverTimestamp).

Coleção despesas

Campo	Tipo	Descrição
propriedadeId	string	Propriedade da despesa (obrigatório: as regras dependem dele).
talhaoId	string | null	Talhão, se a despesa for de um talhão específico; senão null.
categoria	string	insumo | mao_de_obra | maquinario | outros
descricao	string	Texto livre. Em recorrentes recebe sufixo "(1/6)", "(2/6)"...
valor	number	Valor em reais.
data	timestamp	Data de competência. Pode ser futura (parcelas agendadas).
notaFiscalUri	string | null	URI da foto local da nota fiscal.
recorrente	boolean?	true apenas na 1ª parcela de uma série mensal.
recorrenciaMeses	number?	Quantos meses a série tem.
criadoEm	timestamp	Momento do lançamento (Timestamp.now()).

Coleção plantios

Campo	Tipo	Descrição
propriedadeId	string	Propriedade do plantio.
talhaoId / talhaoNome	string | null	Talhão (o nome é copiado para exibir sem consulta extra).
tipoChave / tipoNome	string	Cultura do catálogo (ex.: "milho" / "Milho").
diasParaColheita	number	Ciclo da cultura em dias, copiado do catálogo no momento do cadastro.
dataPlantio	timestamp	Data em que foi plantado.
criadoEm	timestamp	Momento do lançamento.

Decisões de modelagem

- **Coleções planas em vez de subcoleções:** facilita consultas por propriedade e por talhão e a exclusão em lote.
- **Desnormalização controlada:** `talhaoNome`, `tipoNome` e `diasParaColheita` são copiados para o plantio. Assim a lista mostra tudo sem consultas extras, e mudar o catálogo não altera plantios antigos.
- **Datas como Timestamp** do Firestore, permitindo ordenar e filtrar por período.
- **Fotos fora do banco:** só a URI local é salva; a imagem fica no armazenamento do aparelho.

3. Regras de segurança (firestore.rules)

Como o app acessa o banco diretamente, qualquer pessoa com a chave do projeto poderia tentar ler dados. As regras impedem isso: **cada usuário só lê e altera os próprios dados**. Tudo o que não está liberado explicitamente é negado.

Funções auxiliares

```
function signedIn() { return request.auth != null; }

// true se o usuário logado é dono da propriedade informada
function ownsProperty(propriedadeId) {
  return signedIn()
    && get(/databases/$(database)/documents/propriedades/$(propriedadeId))
         .data.userId == request.auth.uid;
}
```

Propriedades

Operação	Condição
read	Logado e (documento não existe ou userId == uid). O "não existe" permite getDoc retornar vazio sem erro.
create	O novo documento precisa ter userId == uid (não dá para criar em nome de outra pessoa).
update	O dono atual é o usuário **e** o userId continua o mesmo (não dá para "doar" a propriedade).
delete	Apenas o dono.

Talhões, despesas e plantios

As três coleções seguem a mesma regra: o acesso é liberado apenas se o usuário for **dono da propriedade pai**, verificado com `ownsProperty(...)`, que faz um `get()` no documento da propriedade.

Operação	Condição
read	ownsProperty(resource.data.propriedadeId)
create	ownsProperty(request.resource.data.propriedadeId)
update	Dono da propriedade atual **e** da propriedade de destino (impede mover um registro para a fazenda de outra pessoa).
delete	ownsProperty(resource.data.propriedadeId)

Consequências no código

- Toda consulta precisa filtrar pelo dono. Por isso `listarDespesasPorTalhao` recebe também o `propriedadeId`: sem esse filtro o Firestore não consegue provar que todos os resultados pertencem ao usuário e recusa a consulta.
- Na exclusão em cascata a propriedade é apagada **por último**, porque as regras dos filhos dependem dela existir.
- As regras têm testes automatizados em `tests/firestore-rules/` (emulador do Firestore).

4. Índices (firestore.indexes.json)

Consultas que combinam um filtro de igualdade com ordenação em outro campo exigem um **índice composto**. O projeto declara:

Coleção	Campos	Uso
despesas	propriedadeId ↑, data ↓	Despesas de uma propriedade por data
despesas	talhaoId ↑, data ↓	Despesas de um talhão por data
propriedades	userId ↑, criadoEm ↓	Propriedades do usuário
talhoes	propriedadeId ↑, criadoEm ↓	Talhões da propriedade
plantios	propriedadeId ↑, dataPlantio ↓	Plantios da propriedade

Na prática, os serviços fazem apenas o `where` no Firestore e **ordenam no cliente** (com `sort`). Isso evita que o app quebre caso os índices ainda não tenham sido publicados. Para publicar regras e índices: `firebase deploy --only firestore`.

5. Autenticação e conta

Login, cadastro e sessão

- **Cadastro** (`Screens/CriarConta.tsx`): `createUserWithEmailAndPassword(auth, email, senha)`, com validação de e-mail e medidor de força da senha.
- **Login** (`Screens/Login.tsx`): `signInWithEmailAndPassword`. Os códigos de erro do Firebase são traduzidos (ex.: `auth/invalid-credential` → "E-mail ou senha incorretos").
- **Recuperar senha:** `sendPasswordResetEmail`.
- **Biometria:** com `expo-local-authentication` o app verifica se há hardware e digital cadastrada, e se a preferência está ligada; então permite reabrir uma sessão já salva com a impressão digital.
- **Sessão persistente:** o Auth guarda o token no AsyncStorage. Em `App.tsx`, `onAuthStateChanged` decide se mostra o Login ou as abas.

Services/conta.ts

Função	O que faz
nomeDoUsuario()	Nome curto para saudação: primeiro nome do displayName ou, se não houver, o início do e-mail capitalizado.
alterarNome(nome)	Atualiza o displayName no Firebase Auth (updateProfile).
enviarLinkTrocaSenha()	Envia o e-mail de redefinição de senha para o usuário logado.
excluirConta(senha, aoAvancar?)	Reautentica com a senha (exigência do Firebase), apaga cada propriedade com tudo o que ela tem, remove a pasta local de notas fiscais e, por fim, apaga o usuário do Auth. O callback `aoAvancar` informa o progresso para a tela.

6. Serviços: Propriedades e Talhões

Services/propriedades.ts

Função	O que faz
criarPropriedade(dados)	addDoc em "propriedades" com criadoEm = serverTimestamp(). Invalida o cache do resumo. Retorna o ID.
listarPropriedades(userId)	Consulta where("userId", "==", userId) e ordena por nome (localeCompare pt-BR).
buscarPropriedade(id)	getDoc; retorna null se não existir.
atualizarPropriedade(id, dados)	updateDoc parcial + invalida o cache.
excluirPropriedade(id)	Exclusão em cascata (abaixo).

Exclusão em cascata

O Firestore não apaga "filhos" automaticamente. A função faz isso manualmente:

```
// 1. Busca em paralelo tudo que pertence à propriedade
const [talhoes, despesas, plantios] = await Promise.all([
  getDocs(query(collection(db,'talhoes'),  where('propriedadeId','==',id))),
  getDocs(query(collection(db,'despesas'), where('propriedadeId','==',id))),
  getDocs(query(collection(db,'plantios'), where('propriedadeId','==',id))),
]);
// 2. Lista de referências, com a PROPRIEDADE POR ÚLTIMO (as regras dependem dela)
const allDocs = [...talhoes, ...despesas, ...plantios, doc(db,'propriedades',id)];
```

```
// 3. Apaga em lotes de 400 (limite do writeBatch é 500 operações)
for (let i = 0; i < allDocs.length; i += 400) {
  const batch = writeBatch(db);
  allDocs.slice(i, i + 400).forEach((ref) => batch.delete(ref));
  await batch.commit();
}
// 4. Invalida o cache e apaga a foto local
```

Services/talhoes.ts

Função	O que faz
criarTalhao(dados)	addDoc com criadoEm = serverTimestamp().
listarTalhoes(propriedadeId)	Talhões da propriedade, ordenados por nome com ordenação numérica ("Talhão 2" antes de "Talhão 10").
atualizarTalhao(id, dados)	updateDoc parcial.
excluirTalhao(id)	deleteDoc.

Validação de área (em `Screens/TalhaoForm.tsx`): antes de salvar, soma as áreas dos talhões existentes, troca a área antiga (em edição) pela nova e compara com `areaTotal`. Se ultrapassar, mostra quanto ainda está disponível e não salva.

```
const soma = talhoes.reduce((acc, t) => acc + t.area, 0);
const somaComNovo = soma - areaTalhaoEdicao + areaNumero;
if (somaComNovo > propriedade.areaTotal) { /* alerta: "Disponível: X ha" */ }
```

7. Serviços: Despesas (Services/despesas.ts)

Função	O que faz
criarDespesa(despesa)	addDoc; garante talhaoId = null quando vazio e criadoEm = Timestamp.now(). Invalida o cache.
atualizarDespesa(id, parcial)	updateDoc + invalida o cache.
excluirDespesa(id)	deleteDoc, invalida o cache e apaga a foto local da nota fiscal.
buscarDespesa(id)	getDoc; null se não existir.
listarDespesasPorPropriedade(pid)	where propriedadeId == pid; ordena por data decrescente no cliente. Avisa no console se faltar índice.
listarDespesasPorTalhao(tid, pid)	where propriedadeId == pid **e** talhaoId == tid (o filtro por propriedade é exigido pelas regras).
totalDespesas(lista)	Soma dos valores (ignora valores não numéricos).
totalDespesasMes(lista, ano, mes)	Soma só as despesas do mês/ano informado.
totalDespesasAno(lista, ano)	Soma só as despesas do ano.

Categorias

O tipo `CategoriaDespesa` aceita 4 valores, e `LABELS_CATEGORIA` traduz para exibição: insumo → "Insumo", mao_de_obra → "Mão de obra", maquinario → "Maquinário", outros → "Outros".

Despesas recorrentes

Quando o usuário marca "recorrente" e escolhe N meses, o formulário (`DespesaForm.tsx`) cria **N documentos independentes**, um por mês, a partir de hoje. Cada um recebe a data do seu mês e a descrição com sufixo "(i/N)". Só o primeiro tem `recorrente: true`, e a foto da nota fiscal é anexada só ao primeiro.

```
for (let i = 0; i < totalMeses; i++) {
  const d = new Date(base); d.setMonth(base.getMonth() + i);
  await criarDespesa({ ..., descricao: `${descricao} (${i+1}/${totalMeses})`,
                       data: Timestamp.fromDate(d), recorrente: i === 0,
                       recorrenciaMeses: totalMeses });
}
```

Como as parcelas futuras já existem no banco, as telas e o resumo **só somam despesas com data até hoje**; as demais aparecem como "agendadas".

8. Serviços: Plantios e estimativa de colheita

Função	O que faz
criarPlantio(plantio)	addDoc com criadoEm. Invalida o cache.
listarPlantiosPorPropriedade(pid)	Plantios da propriedade, do mais recente ao mais antigo.
excluirPlantio(id)	deleteDoc + invalida o cache.
estimativaColheita(dataPlantio, dias)	Calcula data prevista, dias restantes e se já está pronto.
textoEstimativa(estimativa)	Transforma o cálculo em texto amigável.

Cálculo da colheita

```
dataPrevista  = dataPlantio + diasParaColheita
diasRestantes = ceil((dataPrevista@00:00 - hoje@00:00) / 1 dia)
pronto        = diasRestantes <= 0
```

As horas são zeradas para que a contagem seja por dias de calendário. O texto segue estas faixas:

Situação	Texto exibido
diasRestantes ≤ 0	"Pronto para colher"
= 1	"Colheita em 1 dia"
< 30	"Colheita em X dias"
≥ 30	"Colheita em ~N meses" (dias / 30, arredondado)

Catálogo de culturas (constants/plantios.ts)

Cultura	Ciclo (dias)	Cultura	Ciclo (dias)	Cultura	Ciclo (dias)
Milho	120	Trigo	110	Tomate	80
Soja	100	Café	270	Batata	90
Feijão	90	Cana-de-açúcar	365	Alface	45
Arroz	120	Mandioca	300	Cenoura	75

9. Resumo e cache

Services/resumo.ts — carregarResumo(userId, forcar?)

Monta todos os números da Home e da aba Resumo:

- Busca as propriedades do usuário e, **em paralelo** (Promise.all), as despesas e plantios de cada uma.
- Descarta despesas com data futura (parcelas agendadas).
- Calcula: área total, gasto do mês, gasto do ano, número de plantios ativos, **próxima colheita** (o plantio com menos dias restantes) e **gastos dos últimos 6 meses** para o gráfico.
- Exporta também `formatarMoeda(v)`, que formata em R$ (pt-BR).

```
interface DadosResumo {
  propriedades; areaTotal; gastoMes; gastoAno; plantiosAtivos;
  proximaColheita: { propriedadeNome, tipoNome, diasRestantes, texto } | null;
  gastosPorMes: { ano, mes, valor }[];   // 6 meses, do mais antigo ao atual
  despesas; plantios;
}
```

Services/cacheResumo.ts

Home e Resumo recarregam toda vez que ganham foco. Para não baixar tudo de novo ao alternar entre as abas, há um cache em memória de **30 segundos**:

- `comCacheResumo(userId, forcar, buscar)`: reaproveita o resultado se for do mesmo usuário, tiver menos de 30 s e `forcar` não estiver ligado (o "puxar para atualizar" força).
- Guarda a **Promise**, não o resultado: duas telas pedindo ao mesmo tempo geram uma só busca.
- Se a busca falhar, o cache é limpo para a próxima tentativa funcionar.
- `invalidarResumo()` é chamado por toda função que cria, edita ou exclui propriedade, despesa ou plantio. Fica num arquivo separado para evitar import circular entre os serviços.

10. Arquivos locais e preferências

Services/imagens.ts

Fotos de propriedade e de notas fiscais **não vão para a nuvem**; ficam na pasta de documentos do app (expo-file-system). Antes de salvar, são redimensionadas e comprimidas com expo-image-manipulator.

Função	O que faz
enviarImagemPropriedade(uri, id)	Redimensiona para 800 px de largura, JPEG 70%, salva em propriedades/{id}.jpg e retorna a URI.
excluirImagemPropriedade(id)	Apaga o arquivo, se existir.
getImagemLocalUri(id)	URI esperada da foto da propriedade.
enviarNotaFiscal(uri, despesaId)	1000 px, JPEG 60%, salva em notas/{despesaId}.jpg.
excluirNotaFiscal(despesaId) / getNotaLocalUri	Apaga / localiza a foto da nota.

O nome do arquivo é o ID do documento no Firestore, o que liga a foto ao registro sem precisar de outra tabela.

Services/preferencias.ts

Preferências do aparelho, guardadas no AsyncStorage sob a chave `@agroplus/preferencias` (não vão para o Firebase): `biometria` (padrão true), `modoEscuro` (padrão false), `pastaPlanilha` e `ocultarValores`. `lerPreferencias()` mescla o salvo com os padrões; `salvarPreferencia(chave, valor)` grava um campo.

11. Exportação para planilha Excel

A pasta `Services/planilha/` gera um arquivo **.xlsx** do zero, sem bibliotecas de Excel: o formato XLSX é um ZIP de arquivos XML, e o projeto monta esses XML e compacta com a biblioteca `fflate`.

Arquivo	Papel
index.ts	Busca os dados no Firestore (propriedades, talhões, plantios, despesas — em paralelo), chama o montador e salva/compartilha. Funções públicas: `compartilharPlanilha(pid?)`, `baixarPlanilha(pid?)`, `confirmarPrimeiroDownload()`, `pastaDeDownload()`, `esquecerPastaDeDownload()`.
relatorio.ts	`montarRelatorio(dados)` transforma os dados em abas: **Resumo**, **Despesas**, **Gastos por mês**, **Plantios** e **Propriedades e talhões**. Separa despesas lançadas e agendadas e calcula o progresso de cada plantio.
xlsx.ts	`gerarXlsx(abas, info)` escreve os XML (planilhas, estilos, mesclas, fórmulas, datas no formato serial do Excel) e gera os bytes do arquivo.

Com `propriedadeId` o relatório é só daquela propriedade; sem ele, de todas. No Android, o arquivo pode ser salvo numa pasta escolhida pelo usuário (lembrada nas preferências) ou compartilhado (WhatsApp, e-mail) com expo-sharing.

12. Tratamento de erros (Services/erros.ts)

`mensagemErro(error, fallback)` detecta falta de conexão (códigos `unavailable`, `auth/network-request-failed`, `failed-precondition` ou mensagens com "network/offline") e devolve "Sem conexão com a internet..."; nos demais casos devolve a mensagem amigável recebida. As telas usam essa função nos blocos catch e exibem o resultado num toast.

13. Fluxos completos (passo a passo)

Cadastrar uma despesa

- Usuário preenche categoria, descrição, valor, data, talhão (opcional), foto e recorrência.
- A tela chama `criarDespesa` (uma vez ou N vezes, se recorrente).
- O Firestore avalia a regra: `ownsProperty(propriedadeId)` lê a propriedade e confere o `userId`.
- Com o ID retornado, a foto da nota é salva localmente como notas/{id}.jpg e a URI é gravada com `atualizarDespesa`.
- `invalidarResumo()` garante que a Home mostre o novo total.

Abrir a Home

- `carregarResumo(uid)` → cache válido? devolve na hora.
- Senão: `listarPropriedades` → para cada uma, despesas e plantios em paralelo → cálculos → guarda no cache.

Excluir a conta

- Reautenticação com a senha → `excluirPropriedade` para cada propriedade (cascata em lotes) → apaga fotos locais → `deleteUser`.

Dados de exemplo

`Services/dadosExemplo.ts` (botão em Configurações, para testes) cria 5 propriedades em Santa Catarina, cada uma com 2 talhões, 3 plantios e 5 despesas, todas marcadas com `exemplo: true`. `apagarPropriedadesExemplo()` consulta `userId == uid AND exemplo == true` e exclui em cascata.